技　术　交　付　文　档

kafka-file-loader

部 署 文 档

Kafka 消息消费 → CSV 分片生成 → 本地备份 → FTP 投递

单进程常驻 • 无数据库依赖 • jar / lib / config 分离部署

文档版本　v1.0.0
适用制品　kafka-file-loader-1.0.0.tar.gz（约 39.5MB）
运行环境　JDK 1.8 / Linux x86_64 或 Windows Server
编制日期　2026-10-23

目　录

文档说明 1

一、环境要求 1

1.1 软硬件 1

1.2 网络连通性（上线前必须打通） 1

1.3 端口占用检查 2

二、部署包结构 2

三、部署前检查清单 3

四、Linux 部署步骤 3

4.1 上传并解压 3

4.2 创建运行账号与目录权限 4

4.3 修改配置（核心步骤） 4

4.4 启动 6

4.5 启动验证（按顺序逐项确认） 6

五、Windows 部署 8

六、运行时目录与磁盘规划 8

七、日常运维操作 9

7.1 启停 9

7.2 运维接口 9

7.3 常用巡检命令 10

7.4 停机前建议流程 10

八、监控与告警 10

九、升级与回滚 11

9.1 升级（保留配置） 11

9.2 回滚 12
9.3 变更配置（不算升级） 12
十、消费组迁移（高风险，务必按步骤） 12
十一、故障排查 13
十二、附录 14
12.1 systemd 托管（推荐生产使用） 14
12.2 JVM 调优参数 15
12.3 上线验收标准 16
12.4 相关文档 16

文档说明

项	内容
服务名	`kafka-file-loader`
版本	1.0.0
制品	`kafka-file-loader-1.0.0.tar.gz`（约 39.5MB）
技术栈	Spring Boot 2.7.18 / JDK 8 / Spring Kafka / commons-net
依赖中间件	Kafka（消费源）、FTP（投递目标）
依赖数据库	**无**。全流程不连库，配置全在 `config/application.yml`
部署形态	单进程常驻，无状态（状态都在本地目录 + Kafka offset）

服务职责一句话：消费 Kafka 主题 → 按 31 字段顺序生成 `\t` 分隔的 CSV（每 5 万行一个分片）→ 本地备份 → 上传 FTP。

一、环境要求

1.1 软硬件

项	最低	建议	说明
操作系统	Linux x86_64（CentOS 7+ / Ubuntu 18.04+）或 Windows Server 2012+	Linux	生产推荐 Linux，脚本能发 SIGTERM 走优雅停机
JDK	1.8	1.8 / 11	编译目标 1.8；**只需 JRE 即可**，`start.sh` 依赖 `jps`（JDK 自带，JRE 没有，会回退 `pgrep`）
CPU	2 核	4 核	吞吐受磁盘 fsync 与 FTP 带宽限制，CPU 不是瓶颈
内存	1GB（默认堆）	4GB	每批 500 条 × 3 并发，堆 2GB 有余量
数据盘	10GB 可用	50GB+ 独立数据盘	备份保留 7 天，可用空间低于 `min-free-disk-mb`(2GB) 会触发背压停消费
Maven	—	—	**部署机不需要 Maven**，构建在开发机完成

1.2 网络连通性（上线前必须打通）

方向	目标	端口	用途	不通的后果
本机 → Kafka broker	`bootstrap-servers` 全部 16 个节点的 IP	9092	拉取消息	启动后反复重连，无数据

方向	目标	端口	用途	不通的后果
本机 → Kafka broker	`bootstrap-servers` 全部 16 个节点的 IP	9092	拉取消息	启动后反复重连，无数据
本机 → FTP 服务端	`ftp.host`	21（控制）+ 被动模式数据端口段	上传文件	文件堆在 `file-pending/`，达 2000 个触发背压
客户端 → 本机	运维网段	8080（可改）	Swagger / 运维接口 / 健康检查	无法运维，不影响数据链路
本机 → NTP	内网 NTP	123	时间同步	文件名时间戳错乱，offset 排查困难

> FTP 是被动模式（`passive-mode: true`），服务端防火墙必须放通其 `pasv_min_port` ~ `pasv_max_port` 整个端口段，只开 21 是不够的。

1.3 端口占用检查

```
ss -lntp | grep -E ':(8080|9092|21)\b'    # 8080 被占则改 server.port 或启动时 --server.port=18090
```

二、部署包结构

解压后即得如下布局（`includeBaseDirectory=false`，没有多余的一层目录）：

```
kafka-file-loader/                  ← 部署目录，下文记作 $APP_HOME
├── kafka-file-loader.jar           业务代码 228KB（无第三方依赖、无配置文件）
├── lib/                            63 个第三方 jar，41MB（靠 -Dloader.path 加载）
├── config/
│   ├── application.yml             全部业务配置 ★部署时改这里
│   └── logback-spring.xml          日志策略
├── bin/
│   ├── start.sh                    Linux 启停脚本
│   └── start.bat                   Windows 启停脚本
├── 更新说明.txt                     变更记录（源自 CHANGELOG.txt）
├── README.md                       功能与配置详解
├── logs/                           运行日志（首次启动自动创建）
└── data/                           业务数据目录（首次启动自动创建，见 5.3）
```

三层分离的意义：**改配置、换依赖版本都不用重新打包**，只重启进程。

三、部署前检查清单

☐ JDK 已安装：`java -version` 输出 1.8.x

☐ `java -version` 与 `which java` 指向同一 JDK（多版本共存时常见坑）

☐ 到 16 个 Kafka broker 的 9092 全通：`telnet 10.100.1.180 9092`

☐ 到 FTP 的 21 及被动端口段通：`ftp <host>` 能登录、`ls` 能列目录

☐ FTP 账号在 `remote-dir` 下有**写 + 改名 + 删除**权限（上传用 `.uploading` 临时名再 rename）

☐ 部署目录所在磁盘可用 ≥ 10GB，且 `data/` 目录可写

☐ 8080 端口未被占用（或已规划其他端口）

☐ 已确认目标 Kafka 主题存在、分区数 ≥ `kafka.consumer.concurrency`

☐ 已确认消费组 `kafka-hive-loader` 是**新组**（若是从其他环境迁移，先看第十节「消费组迁移」）

☐ FTP 密码已通过环境变量或配置文件准备好

☐ 服务器时区为 `Asia/Shanghai`（文件名时间戳与业务时间一致）

四、Linux 部署步骤

4.1 上传并解压

```
# 规划目录，建议独立数据盘
mkdir -p /opt/app && cd /opt/app
# 上传制品
scp kafka-file-loader-1.0.0.tar.gz root@<目标机>:/opt/app/

# 解压（包内无顶层目录，先建好目标目录再解压进去）
mkdir -p /opt/app/kafka-file-loader
tar -xzf kafka-file-loader-1.0.0.tar.gz -C /opt/app/kafka-file-loader
cd /opt/app/kafka-file-loader

# 核对产物
ls -lh kafka-file-loader.jar          # 约 228K
ls lib | wc -l                       # 应为 63
ls config/ bin/
```

4.2 创建运行账号与目录权限

生产环境不要用 root 跑业务进程：

```
useradd -r -s /sbin/nologin kafkaapp || true
mkdir -p /opt/app/kafka-file-loader/{logs,data}
chown -R kafkaapp:kafkaapp /opt/app/kafka-file-loader
chmod 750 /opt/app/kafka-file-loader
chmod +x /opt/app/kafka-file-loader/bin/start.sh
```

> `logs/`、`data/file-stage`、`data/file-pending` 等子目录都会在启动时自动 `createDirectories`，只要**父目录可写**即可，无需手工建全套。

4.3 修改配置（核心步骤）

编辑 `config/application.yml`。**改完必须重启进程才生效**（本版本不支持热重载）。

必改项

配置项	位置	说明	示例
`kafka.cluster.bootstrap-servers`	第 58 行起	现场实际 broker 列表，端口可省略	`-10.100.1.180:9092`
`kafka.cluster.topic`	`kafka.cluster.topic`	消费主题，写错会导致一直无消息	`DISTRIBUTE_SQAJXYRXX_TO`
`ftp.host` / `ftp.port`	`ftp.*`	现场 FTP 地址	`10.100.9.21` / `21`
`ftp.username`	`ftp.*`	FTP 账号	`kafka`
`ftp.remote-dir`	`ftp.*`	远端投递目录，不存在会自动逐级创建	`/kafka/incoming`
`file.system-name`	`file.*`	文件名前缀（系统配置名称）	`DISTRIBUTE`
`server.port`	`server.port`	运维端口，冲突时改	`18090`

FTP 密码（不要写明文进 yml）

配置里写的是占位符 `${FTP_PASSWORD:kafka}`，优先取环境变量：

```
export FTP_PASSWORD='真实密码'          # 交互式
# 或写进 /etc/sysconfig/kafka-file-loader（systemd 用 EnvironmentFile 加载），并 chmod 600
```

> 注意：若服务器上已存在同名环境变量（如 `SERVER_PORT`、`FTP_PASSWORD`），它会盖掉 yml 里的值。端口与预期不一致时先 `env | grep -iE 'server|ftp'` 看一眼。

按需调整项

配置项	默认	何时需要改
`kafka.consumer.concurrency`	3	主题分区数更多时可提高到 = 分区数
`kafka.consumer.group-id`	`kafka-hive-loader`	**一般不要改**，见第十节
`file.records-per-file`	50000	下游要求更小文件时下调
`file.min-free-disk-mb`	2048	数据盘较小可降到 1024，但不要低于 512
`file.max-pending-files`	2000	FTP 慢、积压多时可适当调大
`file.backup-retention-days`	7	磁盘紧张可减到 3
`file.idle-flush-ms`	60000	低流量要求更实时可降到 10000
`logging.level.com.example.kafkaloader`	DEBUG	长期运行日志量大时改 INFO（省磁盘）
`ftp.enabled`	true	首次联调怕污染 FTP 可先设 false，只落 pending

目录位置建议

默认 `data/`、`logs/` 都是 `./` 相对路径（相对 `start.sh` 切过去的 `$APP_HOME`）。若数据要单独放数据盘，改成绝对路径：

```
file:
  stage-dir: /data/kafka-loader/file-stage
  pending-dir: /data/kafka-loader/file-pending
  backup-dir: /data/kafka-loader/file-backup
  failed-dir: /data/kafka-loader/file-failed
  reject-dir: /data/kafka-loader/file-reject
logging:
  file:
    path: /data/kafka-loader/logs
```

改完记得 `chown -R kafkaapp:kafkaapp /data/kafka-loader`。

4.4 启动

```
cd /opt/app/kafka-file-loader

# 前台首启（推荐首次部署用它，能直接看到失败原因）
su -s /bin/bash kafkaapp -c "FTP_PASSWORD='xxx' java -Dfile.encoding=UTF-8 \
  -Dloader.path=lib -Dlogging.config=config/logback-spring.xml \
  -jar kafka-file-loader.jar"

# 确认无误后 Ctrl+C 停掉，改为后台
su -s /bin/bash kafkaapp -c "FTP_PASSWORD='xxx' ./bin/start.sh start"
```

`start.sh` 支持的子命令：

```
./bin/start.sh start                      # 后台启动
./bin/start.sh start --server.port=18090   # 透传任意 Spring 参数（需要时）
./bin/start.sh status                     # 显示真实 PID 与实际端口
./bin/start.sh restart                    # 改完 config 后用它
./bin/start.sh stop                       # 优雅停机：等批次完成并封盘
```

脚本实际执行的命令（等价，可直接手工执行）：

```
java -Dfile.encoding=UTF-8 \
     -Dloader.path=lib \
     -Dlogging.config=config/logback-spring.xml \
     -jar kafka-file-loader.jar \
     --logging.file.path=logs
```

> `-Dloader.path` 不能省，否则 PropertiesLauncher 找不到 `lib/` 会 `ClassNotFoundException`。`--server.port` 这类参数是**程序参数**，必须放在 `-jar xxx.jar` 之后；JVM 参数（`-Xmx`）不能通过 `start.sh` 透传，用 `JAVA_TOOL_OPTIONS` 或直接改脚本。

4.5 启动验证（按顺序逐项确认）

① 进程存在

```
./bin/start.sh status
# [RUNNING] PID=12345  目录=/opt/app/kafka-file-loader  端口=8080
jps -l | grep kafka-file-loader
```

② 日志出现两条关键行

```
tail -f logs/kafka-file-loader.log
```

必须看到（首次启动约 5~10 秒内）：

```
已装载 Kafka 集群配置 | cluster=DISTRIBUTE | topic=DISTRIBUTE_SQAJXYRXX_TO | 节点来源=application.yml | 节点数=16 | brokers=[...]
分片文件目录 | stage=./data/file-stage | pending=./data/file-pending | reject=./data/file-reject | 每文件 50000 行 | 每行 31 列 | 文件名 DISTRIBUTE_时间戳.csv
```

`节点数=16` 与实际配置一致才算节点解析成功。若为 0 或启动直接失败，说明 `bootstrap-servers` 全部非法。

③ 状态接口

```
curl -s http://127.0.0.1:8080/api/admin/status | python -m json.tool
```

关注返回里的 `nodeCount`（应为 16）、`nodeSource`（固定为 `application.yml (kafka.cluster.bootstrap-servers)`）、`pendingFiles`、`diskFreeMb`、ftp 连通情况。

④ 数据链路冒烟

```
# 手动封盘，把内存里未满 5 万行的分片落盘
curl -X POST http://127.0.0.1:8080/api/admin/files/flush
# {"sealedFiles":1,"pendingFiles":1,"message":"已封盘 1 个分片"}

# 手动备份 + 上传，立即验证 FTP 是否真的通
curl -X POST http://127.0.0.1:8080/api/admin/files/upload
#
{"executed":true,"scanned":1,"uploaded":1,"failed":0,"backupSkipped":0,"elapsedMs":820}
```

`uploaded=1 / failed=0` 即链路通。再核对：

```
ls data/file-backup/dt=$(date +%F)/            # 本地备份应存在同名文件
ls data/file-pending/                          # 上传成功后应为空
# FTP 端确认（-p 为被动模式）
ftp -p <ftp.host>                              # 输入 username/password 后：
ls  /kafka/incoming                                                  # 应看到 DISTRIBUTE_yyyyMMddHHmmssSSS.csv
```

⑤ 文件列数核对

```
awk -F'\t' '{print NF; exit}' data/file-backup/dt=$(date +%F)/DISTRIBUTE_*.csv
# 必须输出 31
```

⑥ 文档页（可选）

浏览器打开 `http://<服务器 IP>:8080/swagger-ui.html`。

五、Windows 部署

```
:: 1. 解压到如 D:\app\kafka-file-loader
:: 2. 改 config\application.yml（同 4.3）
:: 3. 前台启动（关窗口即停，生产建议注册为服务）
set FTP_PASSWORD=真实密码
D:\app\kafka-file-loader\bin\start.bat
```

Windows 注意点：

- `start.bat` 是**前台运行**，输出直接打在控制台，没有 stop 子命令，`Ctrl+C` 或关窗口即停。
- 关窗口属于**强制结束**，不会走优雅停机；残留的 `.writing` 文件由下次启动的 `recover-on-start` 恢复成 pending，**不丢数据**。
- 不需要常驻运维时，用 `nssm install kafka-file-loader "D:\...\java.exe" "-Dfile.encoding=UTF-8 -Dloader.path=D:\app\kafka-file-loader\lib -Dlogging.config=D:\app\kafka-file-loader\config\logback-spring.xml -jar D:\app\kafka-file-loader\kafka-file-loader.jar"` 注册为服务，并在 nssm 里设置环境变量 `FTP_PASSWORD`。
- 路径里不要有中文和空格，`-Dloader.path` 含空格时需要整体加引号。

六、运行时目录与磁盘规划

启动后自动创建（无需手工建）：

```
$APP_HOME/data/
├── file-stage/<topic>/*.writing          正在写入，临时名，未封盘前对外不可见
├── file-pending/*.csv                    已封盘，等待备份+上传（上传成功才删除）
├── file-backup/dt=YYYY-MM-DD/*.csv       本地备份，保留 7 天，上传成功后仍保留
├── file-failed/*.csv + *.reason          重试 10 次仍失败，需人工处理
└── file-reject/DISTRIBUTE_reject_yyyyMMdd.csv   非 JSON 对象的脏数据，按天归档
```

磁盘估算：

目录	峰值估算	清理方式
`file-stage`	约等于「并发数 × 5 万行」	封盘即转 pending，自动

目录	峰值估算	清理方式
`file-stage`	约等于「并发数 × 5 万行」	封盘即转 pending，自动
`file-pending`	取决于 FTP 是否跟得上，上限 `max-pending-files`(2000) 个文件	上传成功即删，自动
`file-backup`	日增量 × 7 天	每天 03:30 cron 清理超期，自动
`file-failed`	正常应为 0	**人工**，处理完手工删除
`file-reject`	正常极小，脏数据多时增长快	**人工**，按天文件自行归档
`logs`	单文件 200MB 切分，总量上限 20GB，保留 15 天	自动

至少要保证：数据盘可用空间 > `min-free-disk-mb` + 一个备份周期增量，否则会触发背压停消费（这是保护机制，不是故障）。

七、日常运维操作

7.1 启停

操作	命令	说明
启动	`./bin/start.sh start`	已在运行会提示并退出，不会起第二个实例
停止	`./bin/start.sh stop`	发 SIGTERM，最多等 60s 让批次完成并封盘，超时再 `kill -9`
重启	`./bin/start.sh restart`	改配置后用它
状态	`./bin/start.sh status`	PID + 部署目录 + 实际端口

> `stop` 不用 `$!` 记录 PID，而是用 `jps`/`pgrep` 反查命令行含 `kafka-file-loader.jar` 的真实 JVM 进程，避免 Git Bash/Cygwin 环境下出现「提示已停止但进程还在」。

7.2 运维接口

方法	路径	用途	使用时机
POST	`/api/admin/files/flush`	立即封盘当前分片	低峰期想让数据尽快出去；停机前
POST	`/api/admin/files/upload`	立即备份+上传（与定时任务互斥）	FTP 恢复后立即补传，不用等 60s 周期
GET	`/api/admin/files/stats`	各目录文件数 + 磁盘剩余	排查积压
GET	`/api/admin/schema`	31 字段名/类型/顺序	下游建表核对

方法	路径	用途	使用时机
POST	`/api/admin/files/flush`	立即封盘当前分片	低峰期想让数据尽快出去；停机前
GET	`/api/admin/status`	集群/FTP/管道总览	巡检第一站

```
curl -X POST http://127.0.0.1:8080/api/admin/files/flush
curl -X POST http://127.0.0.1:8080/api/admin/files/upload
curl -s  http://127.0.0.1:8080/api/admin/files/stats
```

7.3 常用巡检命令

```
# 积压情况
curl -s http://127.0.0.1:8080/api/admin/files/stats
ls data/file-pending/*.csv 2>/dev/null | wc -l
du -sh data/file-backup

# 消费 lag（在 Kafka 客户端机器上执行）
kafka-consumer-groups.sh --bootstrap-server 10.100.1.180:9092 \
  --describe --group kafka-hive-loader

# 最近的上传结果
grep -a "定时上传" logs/kafka-file-loader.log | tail -5
grep -a "封盘待上传" logs/kafka-file-loader.log | tail -5
```

7.4 停机前建议流程

```
# 1. 先封盘，把内存数据落成文件
curl -X POST http://127.0.0.1:8080/api/admin/files/flush
# 2. 主动上传一次，减少停机后积压
curl -X POST http://127.0.0.1:8080/api/admin/files/upload
# 3. 优雅停机
./bin/start.sh stop
```

八、监控与告警

健康检查：`GET http://<host>:8080/actuator/health`（进程存活探针，返回`{"status":"UP"}`）。

指标：`GET /actuator/metrics`，关键三项：

指标	含义

`file.chunks.backup`	已备份文件数
`file.chunks.uploaded`	上传成功文件数
`file.chunks.upload_failed`	上传失败次数

建议配置的告警（按优先级）：

#	条件	含义	处置
1	`pendingFiles` 持续上涨并接近 2000	上传追不上，即将触发背压停消费	查 FTP 连通性与带宽
2	`diskFreeMb` < 3 × `min-free-disk-mb`	磁盘逼近低水位	清 `file-backup`/`file-failed`，扩盘
3	`file-failed/` 出现文件	单文件重试 10 次仍失败	看同名 `.reason`，人工补传
4	`stageFiles` 长期不变但 Kafka 有流量	写入异常（权限/磁盘满）	查日志 ERROR
5	`/actuator/health` 非 200	进程挂了	拉起并查 `logs/console.out`
6	消费组 lag 持续增长	消费跟不上或已停消费	查背压日志与分区数
7	`logs/*.log.gz` 数量异常增长	DEBUG 日志量过大	把业务包级别改 INFO

九、升级与回滚

9.1 升级（保留配置）

```
# 1. 停服务（优雅停机）
cd /opt/app/kafka-file-loader && ./bin/start.sh stop

# 2. 备份当前版本（整目录，含 config 与 data）
cd /opt/app && tar -czf backup/kafka-file-loader-$(date +%F-%H%M).tar.gz kafka-file-loader

# 3. 解压新包到临时目录，只替换代码，保留 config 与 data
mkdir -p /opt/app/new && tar -xzf kafka-file-loader-1.1.0.tar.gz -C /opt/app/new
cp -a /opt/app/new/kafka-file-loader.jar /opt/app/kafka-file-loader/
rm -rf /opt/app/kafka-file-loader/lib
cp -a /opt/app/new/lib /opt/app/kafka-file-loader/
cp -a /opt/app/new/bin/. /opt/app/kafka-file-loader/bin/

# 4. 对比配置差异（新版可能新增配置项，缺项会用代码默认值，但建议补上）
diff /opt/app/new/config/application.yml /opt/app/kafka-file-loader/config/application.yml
# 有新增项时手工合并；logback 有变更才覆盖
# cp -a /opt/app/new/config/logback-spring.xml /opt/app/kafka-file-loader/config/

# 5. 启动并验证（回到 4.5）
```

> **不要整目录覆盖升级**：`config/` 是现场配置，`data/` 是待上传数据，被覆盖会丢配置或丢数据。

9.2 回滚

```
./bin/start.sh stop
cd /opt/app
mv kafka-file-loader kafka-file-loader.failed
tar -xzf backup/kafka-file-loader-<时间>.tar.gz      # 解出 kafka-file-loader/
cd kafka-file-loader && ./bin/start.sh start
```

回滚注意：

- **Kafka offset 不回滚**。旧版本从新版本提交的 offset 继续消费，不会重投已处理数据（这是期望行为）。
- 若新版已经生成了文件名/格式不同的文件，回滚后 FTP 上会并存两种格式，需通知下游。
- 升级前如做过 `group-id` 变更，回滚时也要把 `group-id` 改回去。

9.3 变更配置（不算升级）

```
vi config/application.yml
./bin/start.sh restart
tail -f logs/kafka-file-loader.log      # 确认 节点数 / 目录 两条关键行
```

改配置**不会丢正在处理的数据**：停机时残留 `.writing` 会在启动时恢复为 pending。

十、消费组迁移（高风险，务必按步骤）

`kafka.consumer.group-id` 当前是历史值 `kafka-hive-loader`，**默认不要改**。原因：换 group-id 等于换一个全新消费者，在 `auto-offset-reset: earliest` 下会把该主题**全部历史消息重投一遍**，FTP 上会出现大量重复文件。

确需改名时：

```
# 1. 停服务，确认旧组 lag 已为 0
./bin/start.sh stop
kafka-consumer-groups.sh --bootstrap-server 10.100.1.180:9092 \
  --describe --group kafka-hive-loader          # LAG 列应全为 0

# 2. 记录各分区当前 offset（截图或存文件）
kafka-consumer-groups.sh --bootstrap-server 10.100.1.180:9092 \
  --describe --group kafka-hive-loader > /tmp/old-group-offsets.txt

# 3. 改 config/application.yml 的 group-id，启动一次让它建组后停掉
# 4. 把新组 offset 对齐到第 2 步记录的位置
kafka-consumer-groups.sh --bootstrap-server 10.100.1.180:9092 \
  --group <新组名> --topic DISTRIBUTE_SQAJXYRXX_TO \
  --reset-offsets --to-offset <记录的 offset> --execute

# 5. 启动服务，确认 lag 从 0 开始且无重复文件
```

替代方案：接受重复，由下游按业务主键 `md_id` 去重。

十一、故障排查

现象	可能原因	排查 / 处理
启动即退出，日志无输出	缺 `-Dloader.path=lib` → ClassNotFound；或 JDK 版本不对	看 `logs/console.out`；确认 `java -version` 为 1.8
报 `缺少配置: config/application.yml`	工作目录不对	必须在 `$APP_HOME` 下执行，`start.sh` 会自动 `cd`，手工启动要自己 `cd`
启动失败：bootstrap-servers 解析为空	配置格式错误（如写成 `host:abc`）	查 WARN 日志「跳过非法节点」；至少要有 1 个合法节点
起来后一直无数据	topic 名错 / 无权限 / 该主题确实没消息	`kafka-console-consumer.sh` 手动消费同一主题验证
日志 `节点数=16` 但连不上	网络不通或 broker 端口不是 9092	`telnet <broker> 9092`；查 `org.apache.kafka` 调到 DEBUG 看连接错误
一直报 `Broker may not be available`	防火墙 / broker `advertised.listeners` 配置错	与 Kafka 管理员核对
pending 文件不断堆积	FTP 不可达 / 带宽不足	看日志「上传失败」原因；`curl -X POST .../upload` 手动重试
FTP 上传失败 `425` / 卡住	被动模式端口段未放通	放通 `pasv_min_port~max_port`
FTP 远端文件名乱码	服务端编码非 UTF-8	改 `ftp.encoding`（如 `GBK`）与服务端一致
`failed/` 出现文件	单文件重试 10 次未成功	读同名 `.reason`；网络恢复后手工传或改回 pending

现象	可能原因	排查 / 处理
启动即退出，日志无输出	缺 `-Dloader.path=lib` → ClassNotFound；或 JDK 版本不对	看 `logs/console.out`；确认 `java -version` 为 1.8
消费停止但进程在	**触发了背压**（pending > 2000 或磁盘 < 2GB）	日志搜「背压」；清理磁盘或修 FTP，恢复后自动继续
磁盘写满	备份保留过长 / failed 堆积 / 日志占满	清 `file-failed`、`file-reject`，调小 `backup-retention-days`
文件名时间戳与预期差 8 小时	服务器时区不是 `Asia/Shanghai`	`timedatectl set-timezone Asia/Shanghai`（需重启进程）
`stop` 后进程还在	`jps`/`pgrep` 不可用且 PID 被复用	`ps -ef \`
端口不是配置里的值	环境变量 `SERVER_PORT` 覆盖了 yml	`env \`
下游发现列错位	某列值内混入制表符（已被替换为空格，理论上不会）或下游用了逗号分隔	下游按 `\t` 切分；`awk -F'\t' '{print NF}'` 核对 31 列
主文件里出现脏数据	上游发了非 JSON 对象报文	检查 `file-reject/` 中的原始报文，联系上游修正

日志排查速查：

```
tail -f logs/kafka-file-loader.log                # 实时
grep -a "ERROR" logs/kafka-file-loader.log | tail   # 错误
grep -a "背压" logs/kafka-file-loader.log | tail -5
grep -a "上传失败\|上传" logs/kafka-file-loader.log | tail -10
zcat logs/kafka-file-loader.2026-10-22.*.log.gz | grep -a "ERROR"
cat logs/console.out                              # logback 就绪前的早期异常
```

十二、附录

12.1 systemd 托管（推荐生产使用）

`/etc/systemd/system/kafka-file-loader.service`：

```
[Unit]
Description=kafka-file-loader
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=kafkaapp
Group=kafkaapp
WorkingDirectory=/opt/app/kafka-file-loader
Environment="FTP_PASSWORD=真实密码"
Environment="JAVA_TOOL_OPTIONS=-Xms1g -Xmx2g -Dfile.encoding=UTF-8"
ExecStart=/usr/bin/java -Dloader.path=/opt/app/kafka-file-loader/lib \
  -Dlogging.config=/opt/app/kafka-file-loader/config/logback-spring.xml \
  -jar /opt/app/kafka-file-loader/kafka-file-loader.jar \
  --logging.file.path=/opt/app/kafka-file-loader/logs
# 给足优雅停机时间（Spring 侧 timeout-per-shutdown-phase=60s）
TimeoutStopSec=90
KillSignal=SIGTERM
Restart=on-failure
RestartSec=10
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

```
systemctl daemon-reload
systemctl enable --now kafka-file-loader
systemctl status kafka-file-loader
journalctl -u kafka-file-loader -f
```

> 用 systemd 时**不要再**用 `start.sh start`，两条路会起两个进程。`TimeoutStopSec` 必须大于 60s，否则优雅停机被 systemd 打断，残留分片靠下次启动恢复。

12.2 JVM 调优参数

`start.sh` 未内置堆参数，按需二选一：

```
# 方式一：环境变量（不改脚本，start.bat/sh 都会读到）
export JAVA_TOOL_OPTIONS="-Xms1g -Xmx2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200"

# 方式二：直接改 bin/start.sh 的 java 命令行加参数
```

参考值：小流量（<1000 条/秒）`-Xms512m -Xmx1g`；中等流量 `-Xms1g -Xmx2g`；`max-poll-records` 调到 2000 以上时堆同步上调。

12.3 上线验收标准

#	验收项	通过标准
1	进程	`start.sh status` 显示 RUNNING，PID 稳定 10 分钟不掉
2	启动日志	出现「已装载 Kafka 集群配置」，`节点数` 与实际 broker 数一致
3	状态接口	`/api/admin/status` 的 `nodeCount` 正确，`nodeSource` 为 application.yml
4	消费	消费组 lag 稳定下降，不持续增长
5	生成文件	`data/file-backup/dt=today/` 出现 `DISTRIBUTE_yyyyMMddHHmmssSSS.csv`
6	文件格式	`awk -F'\t' '{print NF}'` 每行均为 31
7	FTP 投递	远端 `remote-dir` 出现同名文件，字节数与本地备份一致
8	备份清理	本地备份只保留 7 天内的 `dt=` 目录
9	日志	按天生成 `.log.gz`，无 ERROR 堆积
10	停机	`stop` 后 60s 内退出，`data/file-stage` 无残留 `.writing`
11	断网演练	断开 FTP 上传 10 分钟，pending 积压但**零丢失**；恢复后自动补传成功
12	重启演练	`restart` 后待上传文件数量前后一致，无重复投递

12.4 相关文档

- `README.md` — 功能设计、31 字段清单、全部配置项详解
- `更新说明.txt` — 版本变更记录
- `docs/loadtest.md` — 四场景压测方案与实测结论
- `docs/benchmark-report.html` — 压测报告